



**INSTITUTO FEDERAL**  
Rio Grande do Sul

---

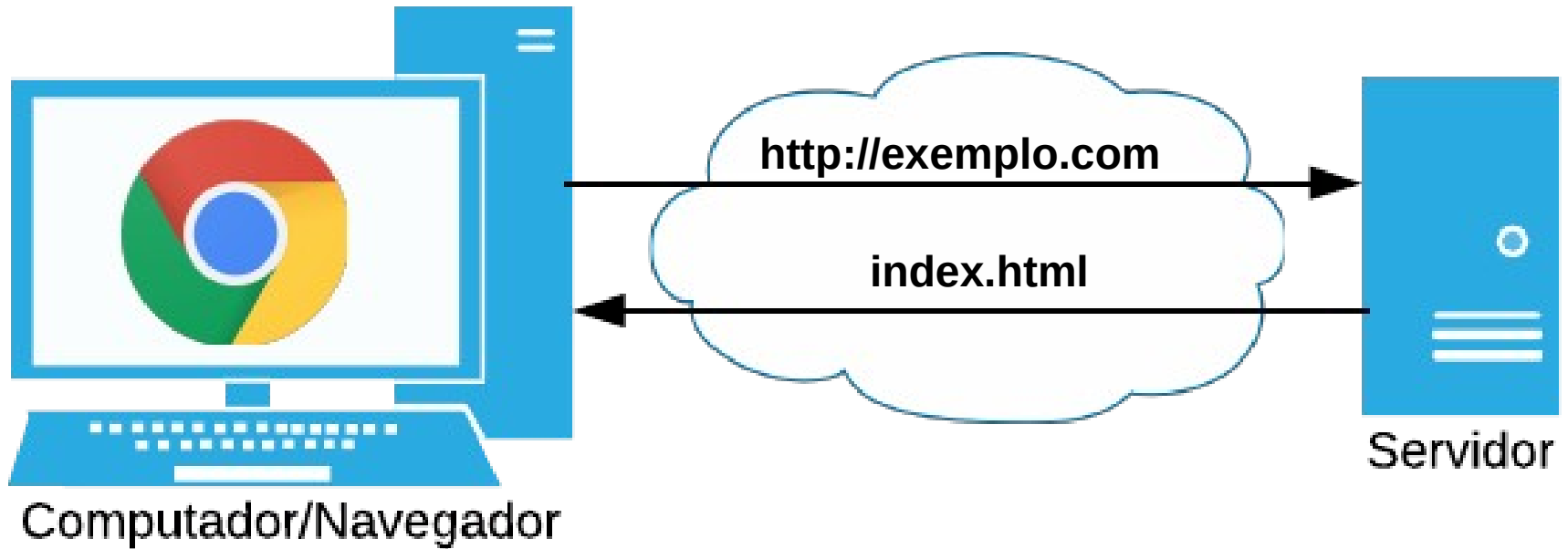
# Front-end

*Programação WEB*

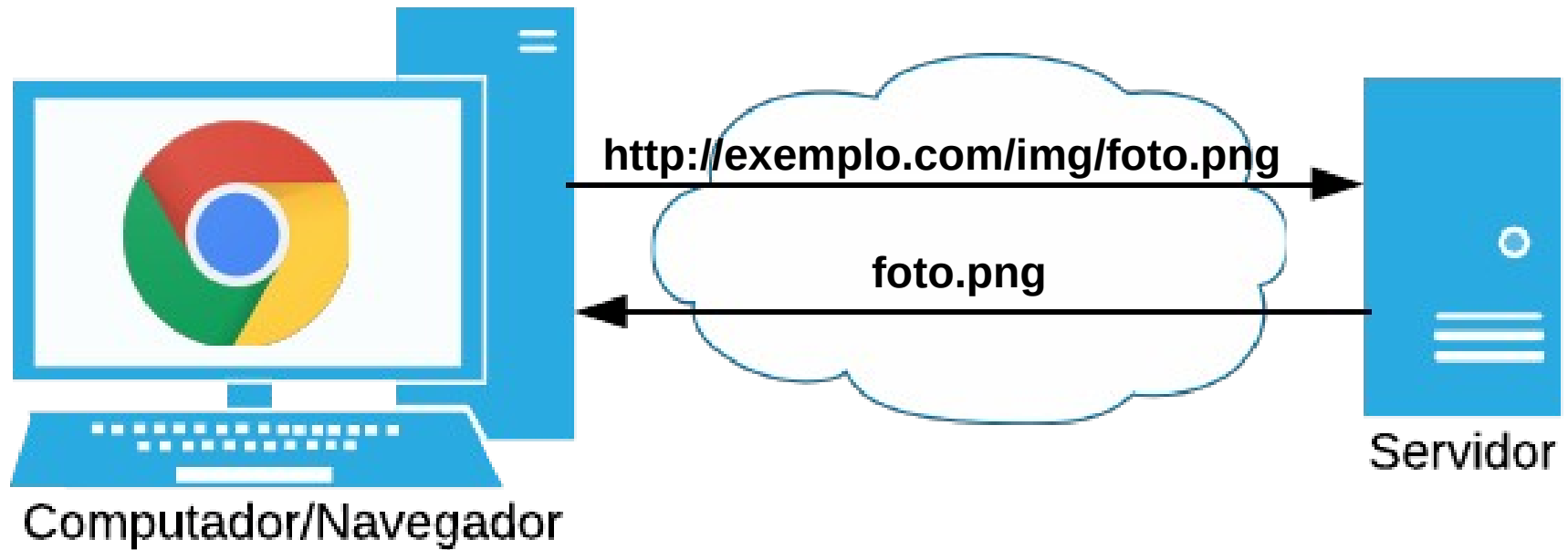
***Professor:** Jezer Machado de Oliveira*



# Programação WEB



# Programação WEB



# Programação WEB

---

Protocolo

Recurso dentro  
do servidor  
(URN)

<http://exemplo.com/img/foto.png>

Endereço  
do servidor  
(URL)



# Programação WEB

---

- **URI (Uniform Resource Identifier)**

- Identificador de Recursos Universal
- Formado pelo Protocolo+URL+URN

- **Protocolo**

- Forma que o servidor e cliente se comunicam

- **URL (Uniform Resource Locator)**

- Localizador de Recurso Universal

- **URN (Uniform Resource Name)**

- Nome de Recurso Universal

# HTTP

---

- **Hypertext Transfer Protocol**

- Protocolo de Transferência de Hipertexto

- Conjunto de regras de transmissão de dados que permitem que máquinas com diferentes configurações se comuniquem

- Protocolo cliente/servidor

- O cliente requisita uma URI e o servidor responde com um recurso

# HTML

---

- **Hypertext Markup Language**

- Linguagem de Marcação de Hipertexto

- **Não é uma linguagem de programação**

- **Serve para definir o conteúdo e a estrutura básica de uma página web**

- **Usam marcações (tag) para criar componentes visuais no navegador**

# Tipos de Sites (Páginas web)

---

- Página Estática
- Página Estática com interação local
- Página Dinâmica no Servidor (Server-Side Rendering – SSR)
- Página Dinâmica no Cliente (Client-Side Rendering – CSR)
- Página Dinâmica no Cliente e Servidor (Web application)



# Página Estática

---

- São páginas prontas, escritas em HTML, CSS e imagens. O servidor apenas entrega o arquivo exatamente como está para o navegador
- Para alterar o site é necessário alterar os arquivos estáticos
- Não permite um comportamento diferente para usuários diferentes

# Página Estática com interação local

---

- São páginas prontas, escritas em HTML, CSS e imagens. O servidor apenas entrega o arquivo exatamente como está para o navegador
- Para alterar o site é necessário alterar os arquivos estáticos
- Não permite um comportamento diferente para usuários diferentes
- Permitem a interação com elementos HTML locais através de uma linguagem de programação

# JavaScript (no Browser)

---

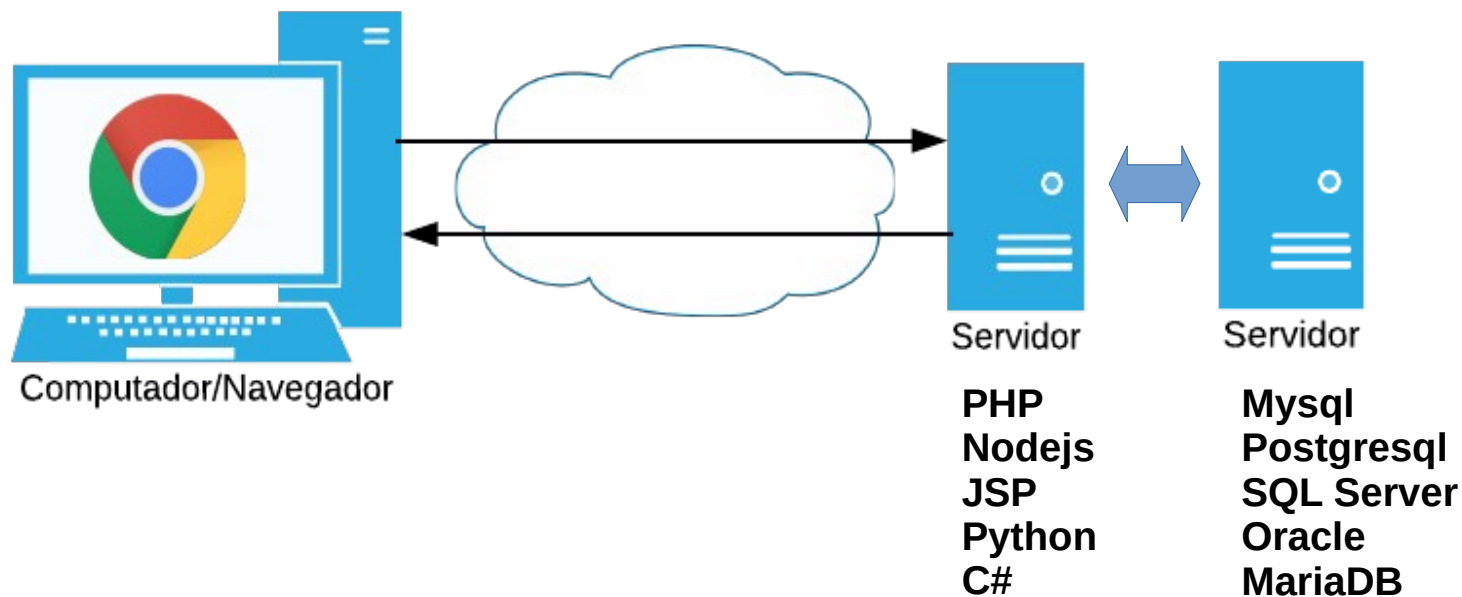
- Linguagem de programação interpretada, criada como uma extensão do HTML
- Oferece recursos que faltam no HTML
- Cria páginas interativas e dinâmicas sem a execução remota de programas no servidor
- Interpretada pelo navegador de internet

# Página Dinâmica no Servidor

---

- O servidor, através de uma linguagem de programação, fornece os arquivos (html, imagens, javascript, css) de forma dinâmica
- O comportamento do site pode mudar com a interação do usuário, tempo, mudanças no banco de dados, etc ...
- Linguagens: PHP, Node.js, JSP, ASP, C#, ....

# Página Dinâmica no Servidor



# Servidor

---

**Código no servidor: exemplo3**

**O resultado é sempre um arquivo estático**

# Página Dinâmica no Servidor

---

- Depois de carregado no navegador do cliente o site só é alterado quando carregado novamente
- O site tem que ser carregado por inteiro para atualizar qualquer parte

# Página Dinâmica no Cliente

---

- O servidor envia uma página “esqueleto” em HTML + arquivos de JavaScript\*, e o navegador monta a interface dinamicamente
- O JavaScript\* no lado cliente manipula e cria os elementos do HTML
- Permite a carga de fragmentos de página e dados à medida que o usuário interage



# Programação Lado Servidor

---

- Todo o código é executado no servidor
- Aceita uma grande gama de linguagens
- Depois de carregada no navegador do cliente a página só é alterada com a interação do usuário
- O página tem que ser carregada por inteiro para atualizar qualquer parte

# Programação Lado Cliente

---

- Boa parte é executado no cliente
- Depende do JavaScript
- Pode alterar qualquer elemento da página sem recarregar
- Depende de uma linguagem no lado do servidor para executar operação nele, por exemplo:  
Acesso ao banco de dados

# Página Dinâmica no Cliente e Servidor

---

- O servidor envia uma página “esqueleto” em HTML + arquivos de JavaScript, e o navegador monta a interface dinamicamente
- O HTML inicial é quase vazio, mas o JavaScript carrega dados de uma API e monta a tela
- O servidor dinâmico fica responsável somente pela entrega de dados sensíveis a mudança (não estáticos)
- Combina tanto interação de programação local, quanto programação no servidor
- Permite um controle quase total das interações e comportamentos oferecidos

# Porque combinar as duas?

---

- Cada uma executa melhor um tarefa
- Melhor interatividade e velocidade
- Redução de carga no servidor
- Cache e otimização
- Separação de camadas

# Single Page Application (SPA)

---

- Todo o site roda em uma única página HTML. O JavaScript manipula o conteúdo sem recarregar a página inteira
- Exemplo:
  - Gmail, Trello, Google Drive
- Características:
  - Navegação fluida (sem reload)
  - Depende fortemente de APIs
  - Precisa de roteamento no cliente

# Páginas Isomórficas (Universal Apps)

---

- Combina as duas abordagens: o servidor já envia uma versão renderizada para o navegador, mas o JavaScript do cliente continua atualizando e interagindo com os dados
- Página abre rápido (renderizada no servidor) e depois fica interativa no cliente
- Características:
  - Boa performance inicial
  - SEO (Search Engine Optimization) amigável
  - Interatividade rica no cliente.

# Progressive Web App (PWA)

---

- Um site que se comporta como aplicativo. Pode ser instalado no celular, funcionar offline e enviar notificações.
- Exemplo:
  - Pinterest, spotify
- Características:
  - Usa Service Workers para cache e offline
  - Experiência próxima a de apps nativos

# Do Ponto de Vista do Programador

---

## ● Programação Front-end (Lado Cliente)

- É a parte da programação responsável pela interface do usuário, tudo aquilo que a pessoa vê e interage diretamente no navegador
- **Exemplo:**
  - Criar o design de uma página, menus, formulários, botões, efeitos de animação
  - Pode trabalhar com uma gama de frameworks, mas fortemente limitado em linguagem de programação



# Do Ponto de Vista do Programador

---

## ● Programação Back-end (Lado Servidor)

- É a parte dinâmica, que roda no servidor. Responsável por processar regras de negócio, manipular banco de dados, autenticação e comunicação com o front-end
- **Exemplo:**
  - Criar as APIs para validar login de usuário, processar pagamentos, salvar e recuperar dados do banco
  - Pode trabalhar com uma gama de framework, linguagem e banco de dados

# Do Ponto de Vista do Programador

---

## ● Programação Back-end

- É a parte dinâmica, que roda no servidor. Responsável por processar regras de negócio, manipular banco de dados, autenticação e comunicação com o front-end
- **Exemplo:**
  - Criar as APIs para validar login de usuário, processar pagamentos, salvar e recuperar dados do banco
  - Pode trabalhar com uma gama de framework, linguagem e banco de dados

# Frameworks e Programação Front-end

- Hoje em dia quase não se fala em programação front-end moderna sem mencionar frameworks ou bibliotecas como **React**, **Angular**, **Vue**, **Svelte** etc.
- Eles não são “obrigatórios” no sentido técnico (é possível programar só com HTML, CSS e JavaScript puro), mas se tornaram necessários no mercado e na prática de desenvolvimento por várias razões.

# Frameworks e Programação Front-end

---

- Complexidade dos sites modernos
- Organização do código
- Produtividade
- Performance e otimização

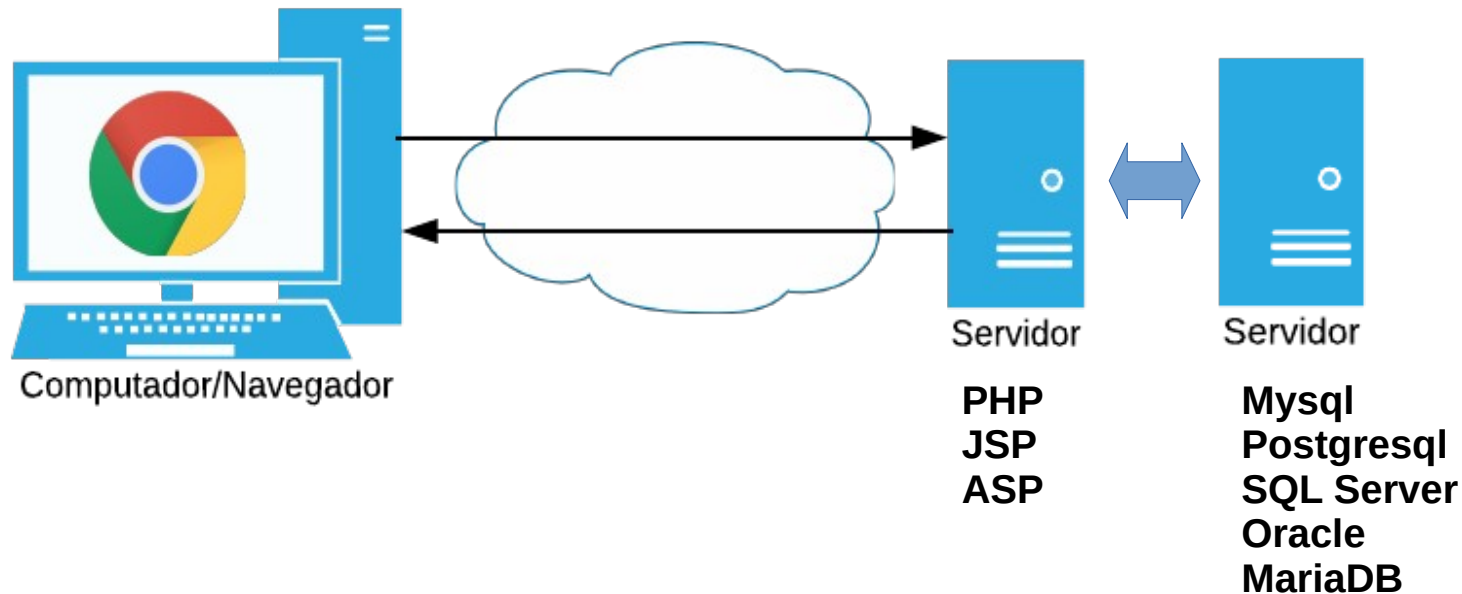
# Proposta

---

## ● Programação Front-end

- HTML
- CSS
- JavaScript
- React

# Programação WEB (~2000)



# Programação WEB (atual)

